

Lab 1: Introduction to EDA and ETL in Data Engineering

A data team typically contains several different roles:

- *data engineer*: maintain the infrastructure and workflows that allow data to move from raw sources into reliable, usable forms
- *data analysts*: build dashboards and business reports
- *data scientists*: explore patterns and develop predictive models
- *machine learning engineers*: deploy and maintain ML systems
- *product/ business teams*: make decisions based on data

In data engineering, raw data often arrives in messy or inconvenient formats. Before it can be used for analytics, machine learning, or reporting, we need to **inspect it, clean it, transform it, and store it** in a more usable form.

In this lab, you will be introduced to 2 key tasks in data engineering:

- **Exploratory Data Analysis (EDA)**: understanding the structure, quality, and patterns of a dataset
- **Extract, Transform, Load (ETL)**: building a simple pipeline to ingest, clean, and store data for downstream use

We will use a flight dataset as the main example and connect EDA with ETL as two parts of the same workflow.

Exploratory Data Analysis (EDA)

EDA helps data engineers to understand

- what the data looks like
- whether the data is complete and trustworthy
- whether the schema is consistent
- whether downstream users can actually use it

A typical EDA process is involved with exploring

- size and shape
- column meanings and types
- missing values
- unusual values or outliers
- distributions
- relationships between variables

Extract, Transform, Load (ETL)

ETL stands for:

- **Extract:** get data from one or more sources
- **Transform:** clean, standardize, enrich, and validate it
- **Load:** store it into a target system for later use

Typical workflow:

1. Extract raw data
2. Inspect structure and sample rows
3. Compute summary statistics
4. Visualize key variables
5. Identify data quality issues
6. Transform data
7. Save cleaned data for downstream use

✓ Exercise: Carrier On-Time Performance Dataset

1. Setup and data loading

Load the load with 29 columns out of 109:

```
[ # 29 columns out of 109 "Year", "Month", "DayofMonth", "DayOfWeek", "DepTime",
"CRSDepTime", "ArrTime", "CRSArrTime",
"Reporting_Airline","Flight_Number_Reporting_Airline", "Tail_Number",
"ActualElapsedTime", "CRSElapsedTime", "AirTime", "ArrDelay", "DepDelay", "Origin",
"Dest", "Distance", "TaxiIn", "TaxiOut", "Cancelled", "CancellationCode", "Diverted",
"CarrierDelay", "WeatherDelay", "NASDelay", "SecurityDelay", "LateAircraftDelay" ]
```

And check the shape of the loaded data

```
import pandas as pd
import numpy as np

## Read data using 29 columns
use_cols = [
    # 29 columns out of 109
    "Year", "Month", "DayofMonth", "DayOfWeek", "DepTime", "CRSDepTime",
    "ArrTime", "CRSArrTime", "Reporting_Airline", "Flight_Number_Reporting_Airline",
    "Tail_Number", "ActualElapsedTime", "CRSElapsedTime", "AirTime", "ArrDelay",
    "DepDelay", "Origin", "Dest", "Distance", "TaxiIn", "TaxiOut", "Cancelled",
    "CancellationCode", "Diverted", "CarrierDelay", "WeatherDelay", "NASDelay",
    "SecurityDelay", "LateAircraftDelay"
]

DATA_DIR = ''
```

```

## TO DO: read_csv()

df = pd.read_csv(DATA_DIR + 'airline_2019.csv', usecols=use_cols)

## Check the shape of the loaded data
df.shape

(76616, 29)

```

✓ 1.1 Rename a few columns:

- "Reporting_Airline" --> "UniqueCarrier"
- "Flight_Number_Reporting_Airline": "FlightNum"
- "Tail_Number": "TailNum"

A useful method here: `df.rename()` method: Original_Name:NewName

```

df.rename(
    columns={

    },
    inplace=True
)

```

Column name convention:

- **Snake Case Convention:** use underscores (_) to separate words
- **Camel Case/ Pascal Case:** capitalize the first letter of each word

1.2 Random sample 20k observations for further analysis

```

## 1.1 Rename a few columns
df.rename(
    columns={
        "Reporting_Airline": "UniqueCarrier",
        "Flight_Number_Reporting_Airline": "FlightNum",
        "Tail_Number": "TailNum"
    },
    inplace=True
)

## 1.2 Random sampling for further analysis
df_sample = df.sample(n=20000, random_state=1)

```

✓ 2. Basic EDA

```
## Print first 5 rows to check how the data looks like  
df.head()
```

	Year	Month	DayofMonth	DayOfWeek	UniqueCarrier	TailNum	FlightNum
0	2019	6	11	2	9E	N927XJ	3459
1	2019	9	2	1	YX	N203JQ	6104
2	2019	3	27	3	OH	N582NN	5125
3	2019	2	13	3	WN	N491WN	172
4	2019	5	8	3	DL	N922DZ	1416

5 rows x 29 columns

```
## Get descriptive statistics and missing value percentage for each column  
def get_info(pd_frame):  
    # generate descriptive statistics (like count, mean, std, min, etc)  
    res = pd_frame.describe().transpose()  
    # calculate the missing percentage for each variable  
    res['missing_percent'] = [(pd_frame.shape[0] - res.loc[ind, 'count']) / pd_frame.shape[0] * 100 for ind in res.index]  
    #  
    res['unique_val'] = [len(pd_frame.loc[:, ind].value_counts()) for ind in res.index]  
    res['var_type'] = ['Continuous' if res.loc[ind, 'unique_val'] >= 10 else 'Categorical' for ind in res.index]  
    return res
```

✓ 3. Visualization and pattern finding

In this part, you need to:

3.1 Visualize the value distribution of each column

3.2 Calculate the missing value percentage of each column

3.3 Visualize the #observations across DayofMonth & DayofWeek

3.4 Visualize arrival and departure delays distribution

`sns.boxplot()` might be a useful function here to visualize spread, outliers, and quartiles.

```
sns.boxplot(data=your_dataframe, x='categorical_column', y='numerical_col')
```

`sns.countplot()` will displays the count of observations for each category in a categorical variable.

```
sns.countplot(data=your_dataframe, x='categorical_column')
```

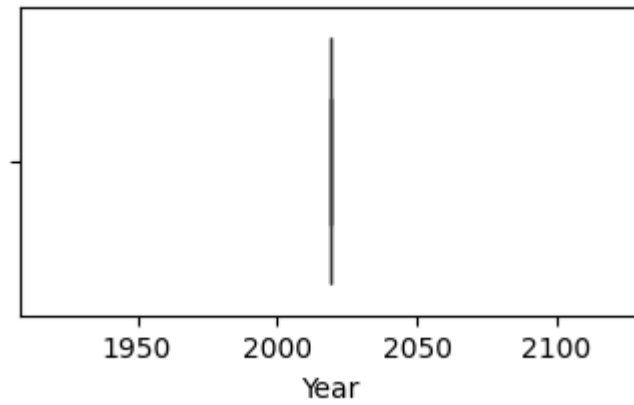
```
## 3.1 Visualize the value distribution of each column
## If the value type is continuous, we will use boxplot
## If the value type is discrete, we will use bar chart

import matplotlib.pyplot as plt
import seaborn as sns
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns

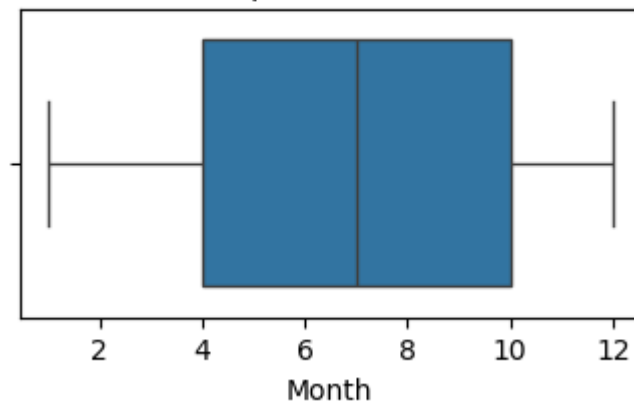
#boxplots
for col in num_cols:
    plt.figure(figsize=(4, 2))
    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col}')
    plt.show()

#bar chart
for col in cat_cols:
    if df[col].nunique() < 20:
        plt.figure(figsize=(4, 2))
        sns.countplot(x=df[col])
        plt.show()
```

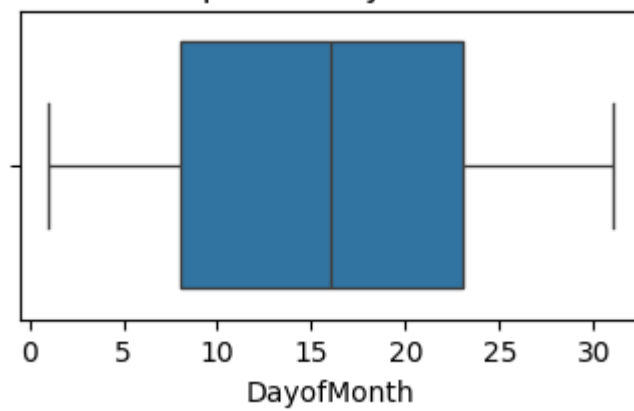

Boxplot of Year



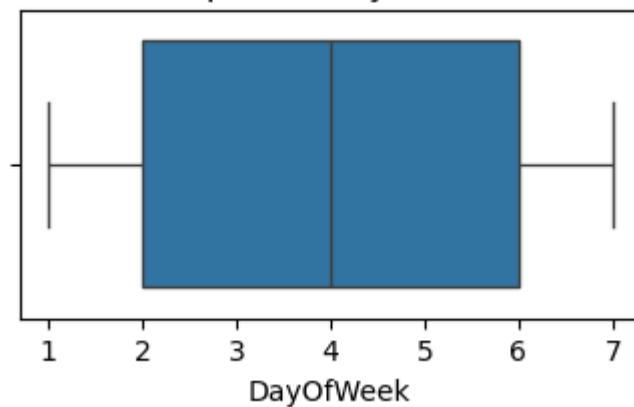
Boxplot of Month



Boxplot of DayofMonth

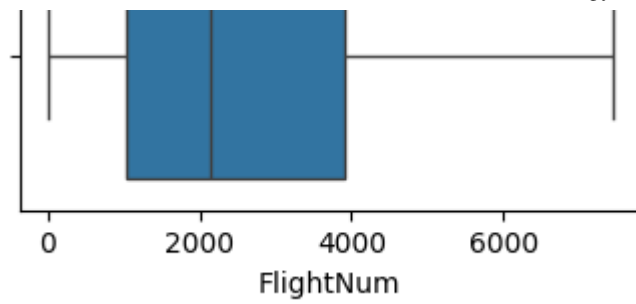


Boxplot of DayOfWeek

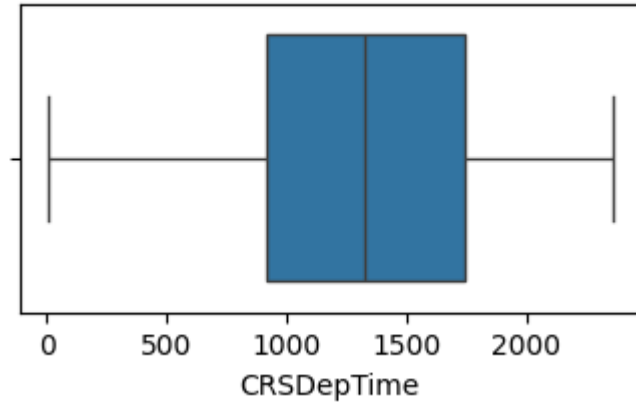


Boxplot of FlightNum

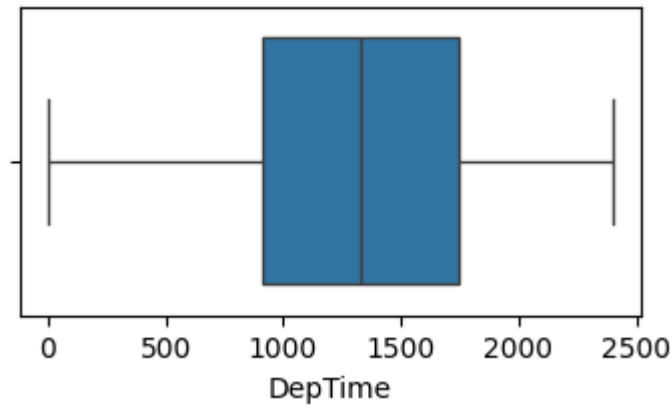




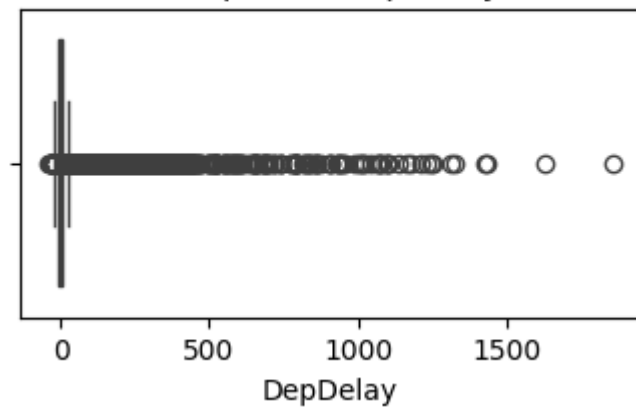
Boxplot of CRSDepTime



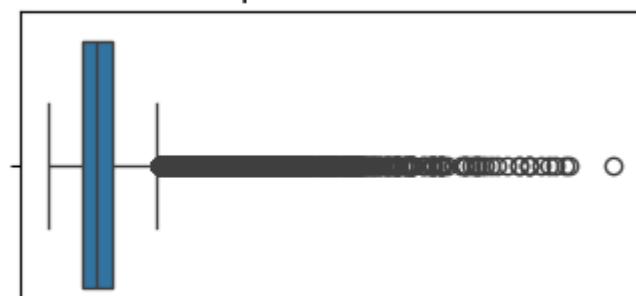
Boxplot of DepTime

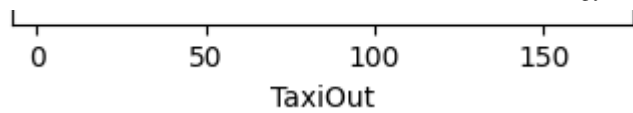


Boxplot of DepDelay

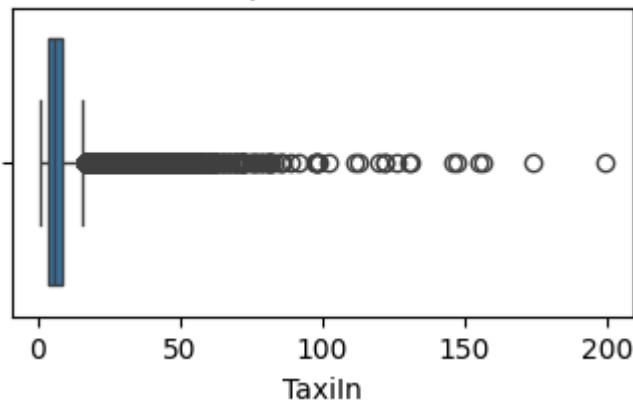


Boxplot of TaxiOut

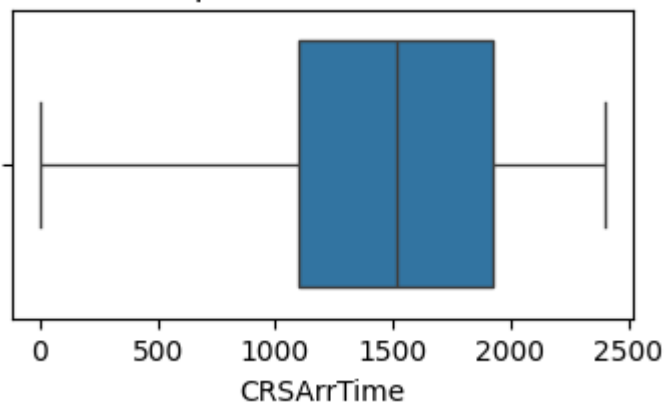




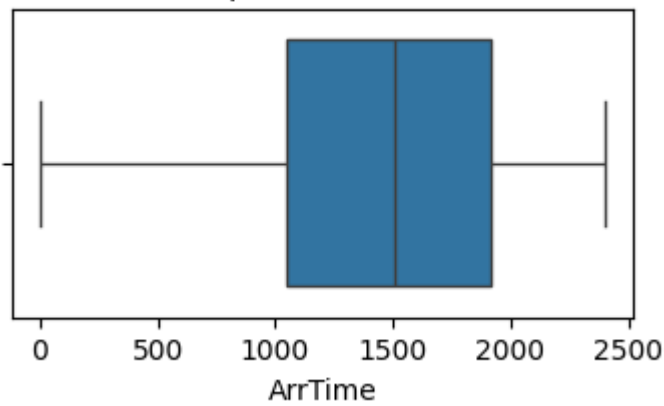
Boxplot of TaxiIn



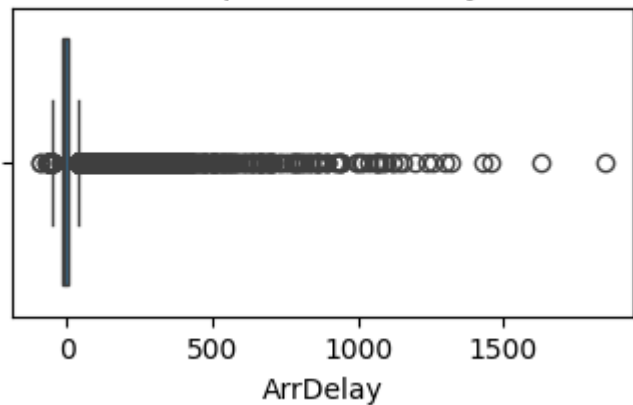
Boxplot of CRSArrTime



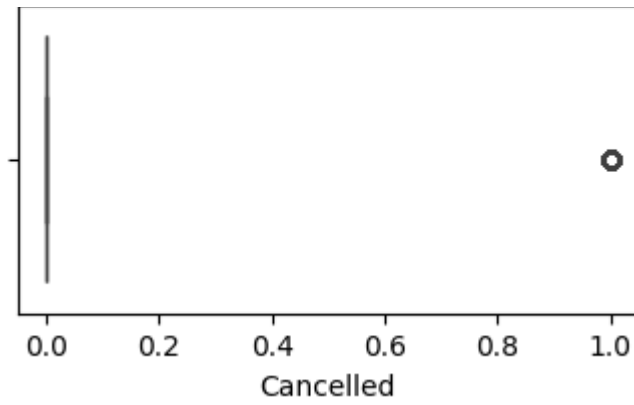
Boxplot of ArrTime



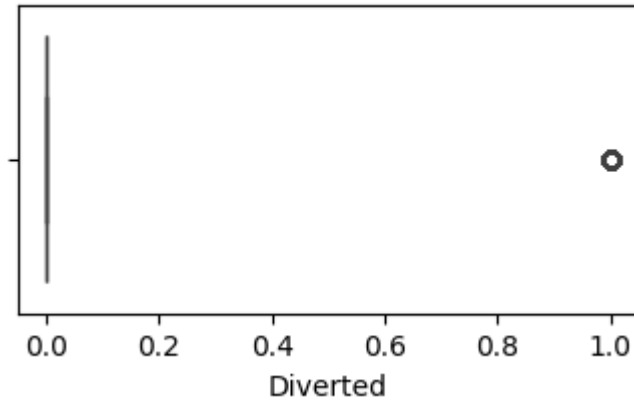
Boxplot of ArrDelay



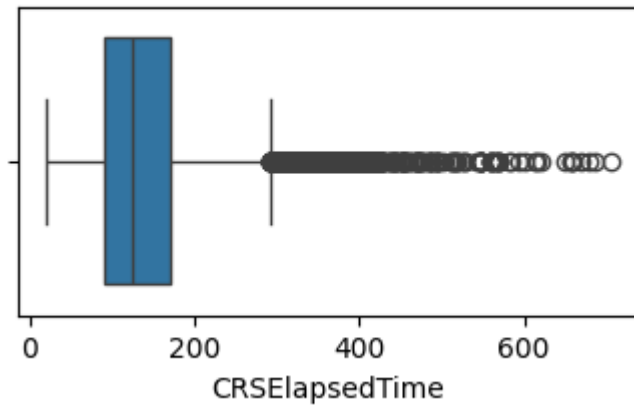
Boxplot of Cancelled



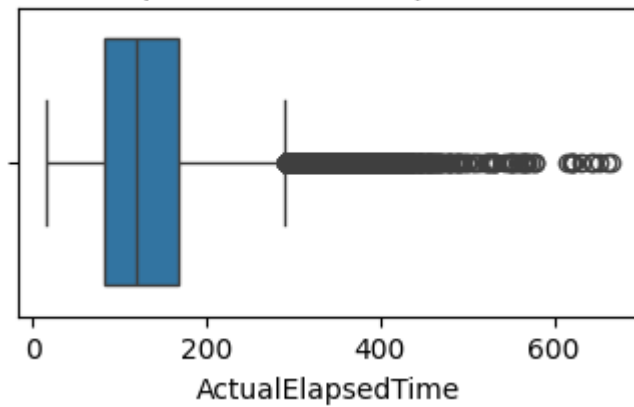
Boxplot of Diverted



Boxplot of CRSElapsedTime

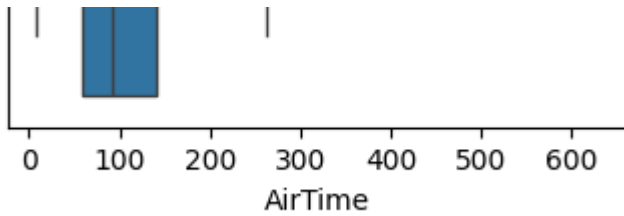


Boxplot of ActualElapsedTime

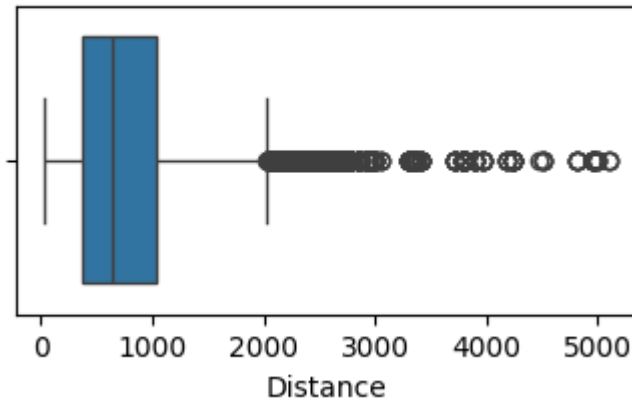


Boxplot of AirTime

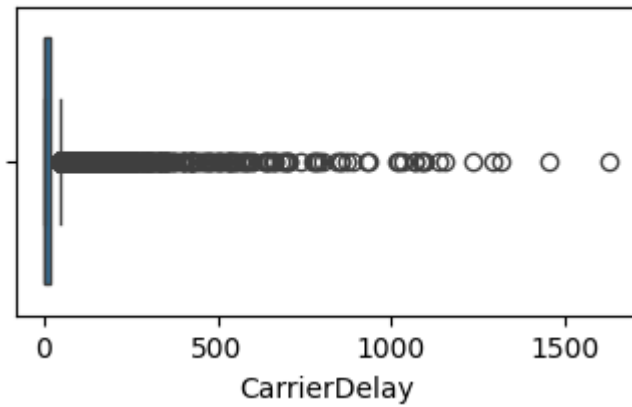




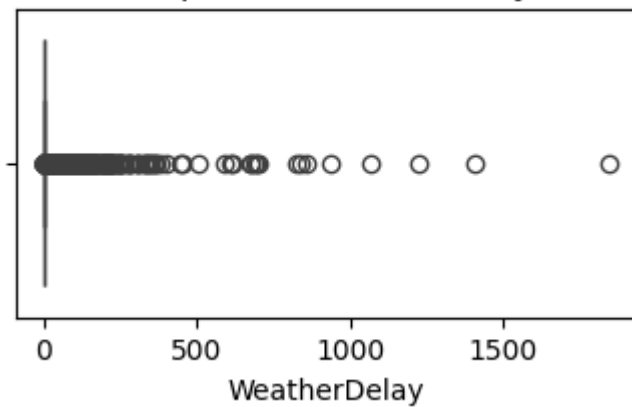
Boxplot of Distance



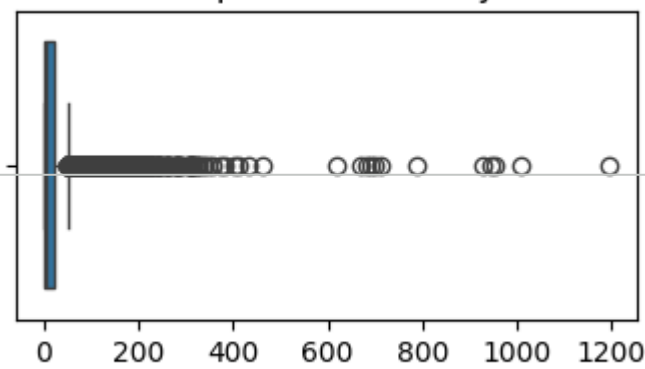
Boxplot of CarrierDelay



Boxplot of WeatherDelay



Boxplot of NASDelay



```
## 3.2 Calculate the missing value percentage of each column  
missing_percent = df.isnull().mean() * 100  
missing_percent
```

